

K-Means Clustering for Customer Segmentation

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain why clustering is an unsupervised learning problem and how it differs from supervised learning
- Walk through the K-means algorithm step by step, including how centroids are chosen and recalculated
- Calculate Euclidean distance and connect it to how K-means decides which centroid a point is "nearest" to
- Implement K-means with scikit-learn and apply it to a customer segmentation example using income and spending data

2. Detailed Explanation

What is clustering? (unsupervised learning in action)

Clustering is the process of grouping similar data points together without any labeled data to guide the grouping. Imagine a dataset of ten students described only by numeric features such as marks. Nobody labels any student as weak, average, or strong ahead of time. Yet a person looking at the numbers can still spot a pattern and sort the students into those categories, using intuition built from past experience. That's the core idea: clustering finds structure in data that was never labeled to begin with.

A second example makes the pattern even clearer. Picture students described by two features: study hours and attendance. Even with no predefined labels, an obvious grouping appears:

- Group one: low study hours plus low attendance.
- Group two: medium study hours plus medium attendance.
- Group three: high study hours plus high attendance.

Nobody assigns these groups in advance — they are discovered directly from the data.

Here's a subtlety worth sitting with: the word "similar" has no fixed, universal definition. Similarity is entirely pattern-based — it depends on the data itself and on the distance measure used, not on a rule stated up front. Points that end up in different clusters are expected to be relatively different from each other.

Customer segmentation: clustering's real-world payoff

Customer segmentation is the business application most closely associated with clustering — it means dividing customers into groups based on similar characteristics or behavior. E-commerce companies group customers who share similar properties, such as age and spending power, so they can send more relevant notifications and coupons to each group. Customers who spend very little, customers who spend moderately, and customers who spend a lot are treated as different segments, with each segment receiving offers suited to its spending pattern. The same idea generalizes beyond customers — any set of entities, such as students, can be segmented the same way. Segmentation is simply clustering applied to a specific business problem, and it's where the algorithmic idea actually becomes useful rather than staying abstract.

Supervised versus unsupervised learning: a quick recap

Before going further into K-means, it helps to place clustering against what you may already know. Supervised learning works with labeled data: every input has a known, corresponding output value (a "Y" value). Unsupervised learning works with unlabeled data, where no such output values exist.

Algorithm	Typical task
Linear regression	Regression, e.g. house price prediction
Logistic regression	Classification, e.g. spam versus not-spam detection
Decision tree	Both classification and regression
Random forest	Both classification and regression (builds on decision trees)

Clustering belongs to unsupervised learning, and K-means is the specific clustering algorithm this lesson focuses on — other clustering algorithms exist but aren't covered here. This lesson builds on a supervised-learning foundation that includes boosting, XGBoost, hyperparameter tuning, and k-fold cross-validation, before shifting into the unsupervised setting.

Breaking down the name "K-means"

The algorithm's name splits neatly into two parts:

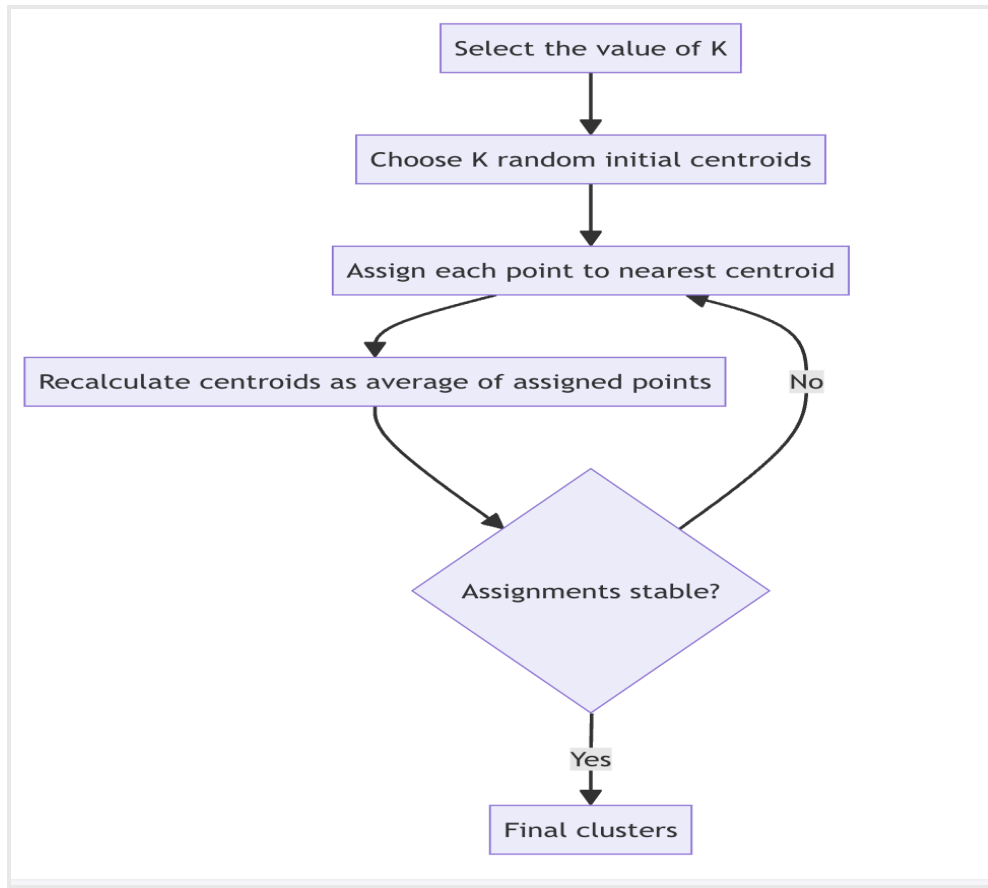
- K refers to the number of groups, or clusters, the algorithm should produce. You choose the value of K before the algorithm starts — setting K equal to 3, for example, asks the algorithm to form three clusters.
- Means refers to the average. K-means uses the average position of the points inside a cluster to represent that entire cluster.

This average position is called the centroid — the center of a cluster, a single point that stands in for every point belonging to it.

The K-means algorithm, step by step

K-means alternates between two moves — assign points to the nearest centroid, then recalculate centroids — until nothing changes anymore:

1. Select the value of K. Decide how many clusters are needed, for example K equal to 3.
2. Choose initial centroids. Pick K random points as the starting centroids (three random points when K equals 3). This mirrors how backpropagation in neural networks starts by initializing the weight vector with random values before learning proceeds — K-means starts the same way, with random initial centroids instead of random weights.
3. Assign each point to its nearest centroid. "Nearest" is what captures the notion of similarity: the point with the least distance to a centroid is assigned to that centroid's cluster. This step produces K clusters out of previously unassigned points.
4. Recalculate the centroids. Once points are assigned to clusters, a new centroid is computed for each cluster as the average of the points currently inside it.
5. Repeat. Steps 3 and 4 repeat, reassigning points to the nearest of the updated centroids and recalculating centroids again, until the assignments stabilize.



Watch out for: since K-means starts from random initial centroids, different runs can settle into different final clusters — this is exactly why scikit-learn's implementation tries multiple random initializations and keeps the best one, covered further below.

Measuring distance: Euclidean distance

K-means needs a formal way to determine how close two points are — this is what "nearest" in step 3 actually means. There are multiple ways to compute distance, and the most commonly used one in K-means is Euclidean distance.

For point one at coordinates (x_1, y_1) and point two at coordinates (x_2, y_2) :

```
distance = sqrt((x2 - x1)^2 + (y2 - y1)^2)
```

Worked example: point A is at $(3, 2)$ and point B is at $(7, 5)$.

- Horizontal difference: $7 - 3 = 4$, squared = 16.
- Vertical difference: $5 - 2 = 3$, squared = 9.
- Sum: $16 + 9 = 25$.
- Distance: $\sqrt{25} = 5$.

Here's the same calculation as a manual implementation using the `math` library:

```
import math

x1 = 3
y1 = 2
x2 = 7
y2 = 5

d = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
print(d)
```

This prints `5.0`, matching the worked example. Euclidean distance is the default distance measure used when nothing else is specified, and it's the one used internally by K-means implementations such as the one in scikit-learn.

Implementing K-means with scikit-learn: imports and exploring the data

Rather than computing distances and centroids by hand, K-means is available as a predefined function inside the scikit-learn library, specifically in its `cluster` module:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

A sample dataset of eight data points is used, each with two features (feature one and feature two). There's no target variable here, because clustering is an unsupervised problem — both columns are features, not a feature-and-label pair.

A scatter plot is the right way to visualize two numeric features, because it shows both the x-coordinate and y-coordinate of every point at once:

```
plt.scatter(data[:, 0], data[:, 1])
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Scatter Plot')
plt.show()
```

`data[:, 0]` selects all rows of the zeroth (first) column, and `data[:, 1]` selects all rows of the first-index (second) column. Looking at the resulting scatter plot makes the grouping visually obvious for this dataset — two clusters stand out clearly, which is why K is set to 2 for this dataset.

Creating, training, and reading results from the model

Configuring a K-means model means choosing a few key parameters:

```
model = KMeans(n_clusters=2, random_state=42, n_init=10)
```

Parameter	What it does
<code>n_clusters</code>	Number of clusters to form — set to 2 here, based on the scatter plot
<code>random_state</code>	Fixes the randomness used to pick initial centroids, so results are reproducible
<code>n_init</code>	Number of different random initializations the algorithm tries, keeping the best result (e.g. 5 or 10)

`n_init` matters because K-means results can depend on where the initial centroids happen to land — running multiple independent attempts and keeping the best guards against a bad random start. You don't need to master how to pick this value in depth; knowing what it does is enough.

Creating the model object does not train it — training happens separately, with `fit`:

```
model.fit(data)
```

`model.fit(data)` runs the select-assign-recalculate cycle from the algorithm steps above until the clusters stabilize. After training, each data point has an assigned cluster label, available as an attribute:

```
labels = model.labels_  
print(labels)
```

`model.labels_` returns, in the same order as the input data, which cluster (0, 1, and so on) each point was assigned to. These numeric labels carry no inherent meaning of their own. Depending on the problem, a practitioner can map them to business-meaningful names, such as treating cluster 0 as "low value" and cluster 1 as "high value." The final centroid positions are available separately:

```
print(model.cluster_centers_)
```

`model.cluster_centers_` returns the x and y coordinates of each cluster's centroid, since a centroid is itself a point with the same number of features as the data.

Plotting clusters with their centroids

The same scatter plot can be extended to show cluster membership by color, with centroids marked distinctly:

```
plt.scatter(data[:, 0], data[:, 1], c=labels)
plt.scatter(
    model.cluster_centers[:, 0],
    model.cluster_centers[:, 1],
    marker='x',
    s=200,
)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Scatter Plot')
plt.show()
```

The `c=labels` argument colors each point according to its assigned cluster, so the two clusters become visually distinguishable. The second `plt.scatter` call plots the centroids on the same figure using an `x` marker instead of the default circular point, with `s=200` enlarging the marker so the centroids stand out against the surrounding data points.

Predicting the cluster of a new data point

A trained model can assign a cluster to a brand-new, previously unseen data point without retraining, using `model.predict()`:

```
pred = model.predict(new_data)
print(pred)
```

A new point placed near the first cluster is predicted as belonging to cluster 0, and a new point placed near the second cluster is predicted as belonging to cluster 1. Multiple new points can also be predicted together in a single call, in which case the returned array gives one label per input point in order.

Watch out for: predicting the cluster for a new point does not, by itself, change the centroids. Centroids only change if the model is retrained — by calling `fit` again — with the new point included in the dataset.

Applying K-means to customer segmentation: a worked example

This is where clustering meets the business problem introduced earlier. A sample dataset includes a customer ID, income, and spending score for each customer. The customer ID is not a feature, since it carries no information about behavior — only income and spending are used as features:

```
X = customers[['income', 'spending']]

model = KMeans(n_clusters=3, random_state=42, n_init=10)
model.fit(X)

print(model.labels_)
```

Three clusters are chosen (K equal to 3), corresponding to low, medium, and high spending power. After training, `model.labels_` returns one cluster label per customer. The result can be visualized the same way as before, plotting income against spending and coloring by cluster:

```
customers['cluster'] = model.labels_

plt.scatter(customers['income'], customers['spending'],
            c=customers['cluster'])
plt.xlabel('Income')
plt.ylabel('Spending')
plt.title('Customer Segmentation')
plt.show()
```

The resulting plot shows three visually distinct groups of customers, corresponding to the three spending-power segments the business can now target separately with tailored campaigns and notifications.

3. Key Takeaways

- Clustering groups similar data points together without any labeled data, relying on patterns rather than a fixed definition of similarity.
- K-means splits into "K" (the chosen number of clusters) and "means" (the average position representing each cluster, called the centroid).
- The algorithm alternates between assigning points to their nearest centroid and recalculating centroids as the average of their assigned points, repeating until the assignments stabilize.
- Euclidean distance, computed as the square root of the sum of squared coordinate differences, is the default way K-means measures "nearest."
- In scikit-learn, `random_state` makes results reproducible, `n_init` runs multiple random initializations and keeps the best outcome, and `model.fit()` trains the model.
- `model.labels_` exposes assigned clusters, `model.cluster_centers_` exposes centroid coordinates, and `model.predict()` assigns clusters to new, unseen points without retraining.
- Customer segmentation — grouping customers by shared characteristics such as income and spending — is the primary practical use case for K-means covered in this lesson.

Mental model: think of K-means as repeatedly asking "who's closest to me?" — every point joins the nearest centroid, every centroid recenters itself on its group, and the process repeats until nobody wants to switch groups anymore.

